

CSS 430 Operating Systems

Program 3A: Synchronization – OS versus HW Supports

1. Purpose

This assignment implements two versions of a producer-consumer synchronization problem: an OS and a HW version. Both versions create a bounded buffer and observe correct data passing over the buffer from a producer to a consumer thread. While hardware (HW) supports, (a.k.a., busy waits or spin loops) are considered faster than OS supports, (i.e., `sem_wait` and `sem_post`) over 2+ CPU cores, program 3A experimentally observes which of HW or OS performs faster synchronization.

2. Producer Consumer Problem.

Review the textbook section 7.1.1 and figures 7.1-7.2. As described in the textbook, the producer executes:

```
wait(empty);
wait(mutex);
/* add next produced to the buffer */
signal(mutex);
signal(full);
```

whereas the consumer executes.

```
wait(full);
wait(mutex);
/* remove an item from buffer to next consumed */
signal(mutex);
signal(empty);
```

The producer's `signal( full )` wakes up the consumer's `wait( full )` upon adding a new data item, while the consumer's `signal( empty )` wakes up the producer's `wait( empty )` upon removing a next data item, which in turn means that a data addition allows the consumer to pick up a new data item and a data deletion creates a space for the producer to add a new data item. Let's define the above two code blocks as `void insert( int item )` and `int remove( )` functions.

If we use Linux OS semaphores, `wait( )` can be replaced with `sem_wait( &empty )`, `sem_wait( &mutex )`, and `sem_wait( &full )`, while `signal( )` can be replaced with `sem_post( &empty )`, `sem_post( &mutex )`, `sem_post( &full )`.

Assuming that the bounded buffer is defined as

```
buf = new int[bufsize];
front = rear = 0;
```

the actual data adding to and retrieval from `buf` will be:

```
/* add next produced to the buffer */ will be buf[ ++rear % bufsize ] = item;
/* remove an item from buffer to next consumed */ will be item = buf[ ++front % bufsize];
```

3. HW Implementation of Semaphores

HW implementations are based on busy waits, which means that a CPU needs to keep checking a shared variable, say `lock` until its content was reset and thus the CPU changed its content. The test-and-set instruction is one of the typical HW supports. C++ can execute this test-and-set CPU instruction at the C++ language level with:

```
atomic_flag_test_and_set_explicit(&lock, memory_order_acquire)
```

or simply with

```
atomic_flag_test_and_set( &lock )
```

Note that `memory_order_acquire` absolutely performs test-and-set through all multi-level cache to DRAM memory.

Look at `3A/semaphore.h`. This header file defines HW-supported semaphore operations: `init( )`, `wait( )`, and `post( )`.

```
using namespace std;
```

```
class Semaphore {
public:
    int init( int *sem, int pshared, int value );
    int wait( int *sem );
    int post( int *sem );
private:
    atomic_flag lock = ATOMIC_FLAG_INIT;
};
```

We simply assume that a semaphore value, (i.e., `sem` in the code) is an integer. Here is an implementation of `init( )`.

```
int Semaphore::init( int *sem, int pshared, int value ) {
```

CSS 430 Operating Systems

Program 3A: Synchronization – OS versus HW Supports

```
*sem = value;
return 0;
}
```

It simply initializes sem to a give value. Note that pshared as the second parameter is nominal just for the purpose of mimicking the Linux OS-supported semaphore's init(). Your first task in 3B is completing semaphore.cpp by adding:

```
int Semaphore::wait( int *sem )
int Semaphore::post( int *sem )
in semmaphre.cpp.
```

Basically, wait() obtains a lock through test-and-set and decrements the sem variable if it's larger than 0. Otherwise, it releases the lock and restarts obtaining the lock. Post() obtains a lock, increments the sem variable, and releases the lock. As we discussed in the class, a HW-suppored semaphore consumes CPU time with busy waits, which deteriorates the execution performance if we have only one CPU. However, 2+ CPUs should work faster. Your task 3 in 3B observes the actual execution performance.

4. Code Structure

The 3A folder includes the following files:

File names	Descriptions
driver.cpp	Receives parameters as follows: ./driver M I P Where M = 1 (HW), = 2 (OS); I = #items to exchange; P = 0: no print, = 1 print Then, main() spawns two threads, each calling producer() and consumer() respectively. Depending on mode (1 == HW or 2 == OS), each thread calls hBuf.insert(i) and oBuf.insert(i) or hBuf.remove(i) and oBuf.remove(i) where hBuf is a bounded buffer with HW semaphores, whereas oBuf is one with OS semaphores. The rest of main() makes sure that each thread sticks to the same CPU, which maintains CPU affinity. No need to change.
semaphore.h	Defines a HW-supported semaphore specification as explained in the above.
semaphore.cpp	Implements a HW-supported semaphore. This is your task 1.
bounded_buffer_os.h	Defines a bounded buffer class that uses Linux OS semaphores. We use three semaphores in sem_t: mutex, full, and empty.
bounded_buffer_hw.h	Defines a bounded buffer class that uses HW-supported semaphores that are defined and implemented in semaphore.h and semaphore.cpp. We instantiate sem_m, sem_f, and sem_e HW semaphores. They correspond to Linux semaphores as shown below: <pre>sem_wait(&full); → sem_f.wait(&full); sem_post(&full); → sem_f.post(&full); sem_wait(&empty); → sem_e.wait(&empty); sem_post(&empty); → sem_e.post(&empty); sem_wait(&mutex); → sem_m.wait(&mutex); sem_post(&mutex); → sem_m.post(&mutex);</pre>
bounded_buffer_os.cpp	Implements a bounded buffer class that uses Linux OS semaphores. As it is a key answer, complete bounded_buffer_os_template.cpp. What you have to do in task 2 is implementing insert() and remove().
bounded_buffer_hw.cpp	Implements a bounded buffer class that uses HW-supported semaphores. As it is a key answer, complete bounded_buffer_hw_template.cpp. What you have to do in task 2 is implementing insert() and remove().
tas.cpp	This is a test-and-set test program that increments the cnt variable from 0 to 99 among a given number of threads in a non-interruptible fashion. It serves as a test-and-set example but is irrelevant to your programming work.
bounded_buffer.c	This is a C version of bounded buffer program with sem_wait() and sem_post(). Not accessible.

5. Statement of Work

Task 1: Implement semaphore.cpp that works as a HW-supported semaphore. Use semaphore_template.cpp and complete wait() and post().

CSS 430 Operating Systems

Program 3A: Synchronization – OS versus HW Supports

Task 2: Implement `bounded_buffer_os.cpp` and `bounded_buffer_hw.cpp`. Use their respective template.cpp file.

Your implementation focuses on `insert()` and `remove()`.

Task 3: Compile and run as follows:

```
[css430@cssmpi1 3A]$ ./compile.sh
[css430@cssmpi1 3A]$ ./driver 1 1000 0
time = 0.0xxxxx seconds
[css430@cssmpi1 3A]$ ./driver 2 1000 0
time = 0.0yyyyy seconds
[css430@cssmpi1 3A]$ ./driver 1 1000000 0
time = z.zzzzzz seconds
[css430@cssmpi1 3A]$ ./driver 2 1000000 0
time = a.aaaaaa seconds
[css430@cssmpi1 3A]$
```

From your measurement, what can you say? Why did it happen. Briefly write your explanation.

6. What to Turn in

	Materials	Points
1	semaphore.cpp a. Code organization: +1pt - Well organized: 1pt, - Poor comments, bad organization, no comments, or horrible code: 1pt b. Correctness: +4pts <code>wait()</code> implemented: +2pt - Correct: 2pts, - Incorrect: 1pt, or - No implementation: 0pts <code>post()</code> implemented: +2pt - Correct: 2pts, - Incorrect: 1pt, or - No implementation: 0pts	5
2	bounded_buffer_os.cpp a. Code organization: +1pt - Well organized: 1pt, - Poor comments, bad organization, no comments, or horrible code: 1pt b. Correctness: +4pts <code>insert()</code> implemented: +2pt - Correct: 2pts, - Incorrect: 1pts, or - No implementation: 0pts <code>remove()</code> implemented: +2pt - Correct: 2pts, - Incorrect: 1pts, or - No implementation: 0pts bounded_buffer_hw.cpp a. Code organization: +1pt - Well organized: 1pt, - Poor comments, bad organization, no comments, or horrible code: 1pt b. Correctness: +4pts <code>insert()</code> implemented: +2pt - Correct: 2pts, - Incorrect: 1pts, or - No implementation: 0pts <code>remove()</code> implemented: +2pt - Correct: 2pts, - Incorrect: 1pts, or - No implementation: 0pts	10
3	An execution snapshot in txt, jpg, png, or pdf - Correct (<code>bounded_buffer_os.cpp</code> runs faster.): 2pts - Wrong: 1pt, or - No results: 0pts	5

CSS 430 Operating Systems
Program 3A: Synchronization – OS versus HW Supports

	Your brief explanations about why OS semaphore runs faster than HW supports while we are using a multi-core machine? • Correct: 3pts • Weak or wrong: 2pts, or • No explanations: 0pts	
--	---	--

Total 20pts.